

HND IN COMPUTING (Videogames Programming)	1VGP	Profesor titular	Pedro Martínez Giménez
Database Management Systems		Profesor asociado	
BASES DE DATOS			

BLOQUE3 / Tema 1: Introducción al Lenguaje SQL

Introducción Lenguaje SQL

- SQL: Structured Query Language, derivado del lenguaje SEQUEL (Structured English Query Language) desarrollado en los laboratorios de IBM en la década de los 70
- Lenguaje para consultar y manipular bases de datos.
- SQL es un Lenguaje de Cuarta Generación (4GL)
- El lenguaje describe lo que debe hacerse, pero no cómo hacerlo, dentro de una sintaxis relativamente simple.
- Cercano al lenguaje natural.
- Adoptado en 1986 por ANSI como estándar.
- Evolución: SQL/86, SQL/89, SQL/92
- SQL NO es sensible a las mayúsculas y minúscula en la estructura de las instrucciones, pero sí en las comparaciones de valores de los campos de las tablas a la hora de realizar filtros en las consultas, inserciones de datos o modificaciones de información.

Grupos Sentencias SQL:

Se pueden agrupar en cuatro tipos de sentencias, cada grupo abarca una serie de instrucciones afines entre sí:

- DDL: Data Definition Language
- DQL: Data Query Language
- DML: Data Manipulation Language
- TCL: Transaction Control Language

Descripción Sentencias SQL:

- DDL: Data Definition Language, Lenguaje de Definición de Datos,
 - El resultado de estas sentencias es un conjunto de tablas, relaciones y reglas cuyas definiciones quedan almacenadas en la BD: metadatos, diccionario de datos o catálogo del sistema.
- DQL: Data Query Language, Lenguaje de Consulta de Datos,
 - Recuperación de Información,
 - Consultas en la Base de Datos
- DML: Data Manipulation Language, Lenguaje de Manipulación de Datos,
 - Inserción de la nueva Información.
 - Eliminación (Borrado) de la información existente.
 - Modificación de la Información almacenada
- TCL: Transaction Control Language, Lenguaje de Control de Transacciones,
 - Lenguaje que se encarga de confirmar o restaurar transacciones de la base de datos.

GRUPO DE INSTRUCCIONES DQL/DML:

- **DQL:**
 - **SELECT:**
 - Seleccionar información de una base de datos

- **DML:**
 - **INSERT**
 - Añade nuevas filas a una tabla

 - **UPDATE**
 - Actualiza filas de una tabla

 - **DELETE**
 - Elimina filas de una tabla

DQL: Consulta de datos con en el Lenguaje SQL

Seleccionar información de una o varias tablas (o vistas),
estructura básica de una instrucción de consulta o
Query:

SELECT * | TablaX.* | Campo1[,
Campo2, ...]
FROM TablaX [, TablaY, ...]

- El asterisco “*” indica seleccionar todos los campos de la(s) tablas.
- “|” Indica que es una opción u otra.
- Los corchetes “[]” indican que no es obligatorio su contenido en la instrucción, solo si se requiere en la consulta, esto quiere decir que, solo si hay más de un campo en la zona del *SELECT* o si hay más de una tabla y/o vista indicada en el *FROM*.

Donde:

- Cláusula *SELECT*, indicamos los campos a visualizar separados por comas o bien usamos el asterisco “*” para seleccionar (visualizar) todos los campos de la(s) Tabla(s)
- Cláusula *FROM*, indicamos la(s) tabla(s) o Vista(s) donde se encuentran los campos a visualizar de la cláusula *SELECT*, si hay más de una tabla las iremos indicando separadas por comas.

Hemos de indicar que “***SELECT***” y “***FROM***” SON PALABRAS RESERVADAS, por lo que no podemos definir tablas, vistas y/o campos con esos nombres.

Ejemplo 1 (Alumnos):

Crear la siguiente tabla e introducir la siguiente información:

- Tabla Alumnos

Nombre del campo	Tipo de datos
NIP	Número
NOMBRE	Texto largo
CALIFICACION	Número
EDAD	Número

- Datos Tabla Alumnos

NIP	NOMBRE	CALIFICACION	EDAD
15230	PEDRO MARTINEZ	5	30
35620	JUAN LOPEZ	6	25
45973	EVA GARCIA	5	36
69981	LUIS MORATA	8	28
78962	CARMEN MARI	4	
91020	ALFREDO	5	36
89654	FRANCISCO	7	28
14230	PEDRO JUAN	5	26
35390	JUAN VARGAS		29
15973	EVA GRACIA	5	32
62981	LUIS FERNANDEZ	8	45
78922	CARMEN MARINA	4	30
89694	FRANCISCO GOMEZ	7	29
91220	ALFREDO L	5	28

Escribid las siguientes sentencias en las BBDD:
SQLite y MariaDB:

- **SELECT** NIP, NOMBRE **FROM** Alumnos
- **SELECT** * **FROM** Alumnos
- **SELECT** CALIFICACION
FROM Alumnos
- **SELECT** Nombre, Calificacion
FROM Alumnos
- **SELECT** Alumnos.* **FROM** Alumnos

AMPLIANDO SINTAXIS SELECT

```

SELECT [predicado] { * | tabla.* | [tabla.]campo1 [AS alias1]
                  [, [tabla.]campo2 [AS alias2] [, ...]] }
FROM expresióndetabla [AS aliastabla ] [, ...]
    
```

Parte	Descripción
<i>Predicado</i>	Uno de los siguientes predicados: ALL, DISTINCT. Utilice el predicado para restringir el número de registros devueltos. Si no se especifica ninguno, el predeterminado es ALL.
*	Especifica que se han seleccionado todos los campos de la tabla o tablas especificadas.
<i>tabla</i>	Nombre de la tabla que contiene los campos de los cuales se seleccionan los registros.
<i>campo1, campo2</i>	Nombres de los campos que contienen los datos que desea recuperar. Si incluye más de un campo, se recuperan en el orden en que se han enumerado.
<i>alias1, alias2</i>	Nombres que se utilizan como encabezados de columna en <i>tabla</i> en vez de los nombres de columna originales.
<i>expresióndetabla</i>	Nombre de la tabla o tablas que contienen los datos que se desean recuperar.
<i>aliastabla</i>	Nombres que se utilizan seudónimo en la <i>tabla</i> en vez de los nombres de tabla original.

Escribir las siguientes sentencias:

- **SELECT** NIP as código, NOMBRE
FROM Alumnos

- **SELECT** ALUMNOS.CALIFICACION as Nota
FROM Alumnos

- **SELECT** A.edad, A.nip as codigo_alumno
FROM Alumnos as A

PREDICADOS EN EL SELECT

SELECT [ALL | DISTINCT] {.....}
FROM TABLA_x

Una instrucción SELECT que contiene estos predicados consta de las siguientes partes:

Parte	Descripción
ALL	Se supone si no se incluye uno de los predicados que selecciona todos los registros que cumplen las condiciones de la instrucción SQL . Los dos ejemplos siguientes son equivalentes y devuelven todos los registros de la tabla Alumnos: SELECT ALL * FROM Alumnos SELECT * FROM Alumnos
DISTINCT	Omite los registros que contienen datos duplicados en los campos seleccionados. Para ser incluidos en los resultados de la consulta, los valores de cada campo enumerado en la instrucción SELECT deben ser únicos. Por ejemplo, varios alumnos incluidos en la tabla Alumnos pueden tener la misma nota. Si varios registros contienen el mismo valor en el campo Nota, la siguiente instrucción SQL devuelve sólo los valores distintos: SELECT DISTINCT Calificacion,Edad FROM Alumnos; Sin Distinct: SELECT Calificacion,Edad FROM Alumnos; Ahora consultamos las distintas edades de la tabla, SELECT DISTINCT Edad FROM Alumnos; Sin Distinct; SELECT Edad FROM Alumnos; Si omite DISTINCT, estas consultas devuelven todas las notas o edades de la tabla Alumnos. Si la cláusula SELECT contiene más de un campo, la combinación de valores de todos los campos debe ser única para que un determinado registro se incluya en los resultados.

	Si la cláusula SELECT contiene más de un campo, la combinación de valores de todos los campos solo se mostrara una sola vez. SELECT DISTINCT Edad, Calificacion FROM Alumnos; SELECT DISTINCT Calificacion, NOMBRE FROM Alumnos; Sin Distinct: SELECT Edad, Calificacion FROM Alumnos; SELECT Calificacion, NOMBRE FROM Alumnos;
LIMIT NumReg “NumReg” es un valor entero positivo y mayor que cero e indica el número de registros máximo que queremos obtener.	Devuelve cierto número de registros que se hallan en la parte superior o inferior de un intervalo especificado por una cláusula ORDER BY. Suponga que desea ver los nombres de los 2 mejores SELECT Nombre, Calificacion FROM Alumnos ORDER BY Calificacion DESC LIMIT 2 Visualizamos las tres edades más bajas: SELECT Edad FROM Alumnos ORDER BY 1 LIMIT 3
Tablax	Nombre de la tabla de la cual se recuperan los registros.

POR LO TANTO, PARA LIMITAR EL NÚMERO DE REGISTROS RECUPERADOS EN LAS BASES DE DATOS DE MARIADB O SQLITE:

SELECT expresiones
FROM tabla(s)
[ORDER BY----
LIMIT NumeroDeFilasARecuperar;

Donde:

“NumeroDeFilasARecuperar” es un valor entero positivo y mayor que cero e indica el número de registros máximo que queremos obtener.

También podemos seleccionar un rango de registros:

```
SELECT column_list  
FROM table  
[ORDER BY ....]  
LIMIT row_count OFFSET offset;
```

O bien,

```
SELECT column_list  
FROM table  
[ORDER BY ....]  
LIMIT offset, row_count;
```

Donde:

- *Row_Count* → Número de registros máximos a obtener de la consulta.
- *Offset* → Indicamos desde que posición de los valores obtenidos en la consulta queremos ver los registros.

Ejemplo:

```
SELECT * FROM Alumnos LIMIT 3 OFFSET 2;
```

O bien

```
SELECT * FROM Alumnos LIMIT 2,3;
```

alumnos (3r × 4c)			
NIP	Nombre	Calificacion	Eddad
15973	EVA GRACIA	5	32
35390	JUAN VARGAS	(NULL)	29
35620	JUAN LOPEZ	6	25

Cláusula FROM

Especifica las tablas o consultas que contienen los campos enumerados en la instrucción SELECT.

Sintaxis

SELECT ListaDeCamposSeparadosPorComas

FROM NombreDeLasTablasSeparadasPorComas

Una instrucción SELECT que contiene una cláusula FROM consta de las siguientes partes:

Parte	Descripción
listadecampos	Nombre del campo o campos que se van a recuperar junto con cualquier alias de nombre de campo, funciones de agregado de SQL, predicados de selección (ALL, DISTINCT) u otras opciones de la instrucción SELECT.
Nombredelastablas	Expresión que identifica una o varias tablas de las que se recuperan datos. La expresión puede ser un solo nombre de tabla, un nombre de consulta guardada o vista.

Comentarios:

Se requiere FROM a continuación de cualquier instrucción SELECT. **La cláusula “FROM” ES OBLIGATORIA PONERLA EN TODAS LAS CONSULTAS SQL.**

El orden en el que se escriben los nombres de las tablas en **expresióndelastablas** no es importante, ni afecta al rendimiento.

Cláusula WHERE

```
SELECT [predicado] { * | tabla.*,  
                  [tabla.]campo1 [AS alias1]  
                  [, [tabla.]campo2 [AS alias2]  
                  [, ...]]}  
FROM expresióndetabla [, ...]  
[WHERE... ]
```

Indica a qué registros de las tablas enumeradas en la cláusula FROM son seleccionadas y afecta a una instrucción SELECT, UPDATE o DELETE.

Una instrucción SELECT que contiene una cláusula WHERE consta de las siguientes partes:

Sintaxis

```
SELECT listadecampos-separadosporcomas  
FROM expresióndetabla(s)-separadasporcomas  
WHERE criterios
```

Donde:

Parte	Descripción
<i>listadecampos</i>	Nombre del campo o campos que se van a recuperar junto con cualquier alias de nombre de campo, predicados de selección (ALL, DISTINCT) u otras opciones de la instrucción SELECT.
<i>expresióndelastablas</i>	Nombre de la tabla o tablas de las cuales se recuperan datos.
<i>criterios</i>	Expresión que deben cumplir los registros para ser incluidos en los resultados de la consulta. Las expresiones se van separando con las conectivas lógicas: AND / OR / NOT

LÓGICA TRIVALUADA [V/F/NULL]

¿Cómo es la condición F?

Tipos de Comparación:

- Nulo(A_i)
- $A_i \alpha A_j$
- $A_i \alpha a$

donde α es un operador de comparación ($<$, $>$, $\leq$, $\geq$, $=$, $<>$), A_i y A_j son nombres de atributo y a es un valor del dominio asociado al atributo A_i distinto del valor nulo.

Las Condiciones se construyen a partir de comparaciones, usando los paréntesis y los operadores lógicos ($\vee$, $\wedge$, $\neg$)

(OR, AND, NOT)

¿Cómo se evalúa la condición $F(t)$?

Valor Nulo $\Rightarrow$ Necesidad de una Lógica Trivaluada $\{V, F, \text{indefinido}\}$:

- Si F es de la forma $A_i \alpha A_j$ entonces $F(t)$ se evalúa a indefinido si al menos uno de los atributos, A_i o A_j tiene valor nulo en t , en caso contrario se evalúa al valor de certeza de la comparación $t(A_i) \alpha t(A_j)$;
- Si F es de la forma $A_i \alpha a$ entonces $F(t)$ se evalúa a indefinido si A_i tiene valor nulo en t , en caso contrario se evalúa al valor de certeza de la comparación $t(A_i) \alpha a$;
- Si F es de la forma $\text{nulo}(A_i)$ entonces $F(t)$ se evalúa a cierto si A_i tiene valor nulo en t , en caso contrario se evalúa a falso.

Lógica Trivaluada:

(Tablas de verdad de las conectivas lógicas $\wedge$, $\vee$, $\neg$)

G	H	AND $F = G \wedge H$	OR $F = G \vee H$
falso	falso	falso	falso
indefinido	falso	falso	indefinido
cierto	falso	falso	cierto
falso	indefinido	falso	indefinido
indefinido	indefinido	indefinido	indefinido
cierto	indefinido	indefinido	cierto
falso	cierto	falso	cierto
indefinido	cierto	indefinido	cierto
cierto	cierto	cierto	cierto

G	NOT $F = \neg G$
falso	cierto
indefinido	indefinido
cierto	falso

Comentarios Cláusula WHERE:

- El motor de base de datos selecciona los registros que cumplen la condición, esto es, **QUE HACEN VERDADERA las condiciones enumeradas en la cláusula WHERE.**
- Si no se especifica una cláusula WHERE, la consulta devuelve todas las filas de la tabla.
- Si se especifica más de una tabla en la consulta y no incluye una cláusula WHERE ni una cláusula JOIN, la consulta genera un producto cartesiano de las tablas, Ejemplo:
- `SELECT * FROM Tabla1,Tabla2`
- `Tabla1 → N_Registros`
- `Tabla2 → M_Registros`
- El Total de registros devueltos por la consulta es un Producto Cartesiano de $(N_Registros \times M_Registros)$.

- WHERE es opcional, pero cuando se incluye, sigue a la Cláusula FROM.

EJEMPLOS:

- **SELECT** NIP, Nombre
FROM Alumnos
WHERE Nip=15230
- **SELECT** *
FROM Alumnos
WHERE Edad<=28
- **SELECT** Calificacion
FROM Alumnos
WHERE Calificacion>6 **OR** Edad=35
- **SELECT** Nombre
FROM Alumnos
WHERE Edad>27 **AND** Calificacion<7
- **SELECT** Nombre
FROM Alumnos
WHERE Edad>30 **AND** Calificacion<6 **OR**
NOMBRE = "ALFREDO"

Cláusula ORDER BY

```
SELECT listadecampo(s),....  
FROM expresióndetabla(s),....  
[WHERE ... ]  
[ORDER BY campo1 [ASC|DESC ],  
             campo2 [ASC|DESC ],  
             Posicionx [ASC|DESC ], .....]
```

- ORDER BY se utiliza para ordenar los resultados de una consulta, según el valor de la(s) columna(s) especificada(s).
- **ORDER BY SIEMPRE SE COLOCA AL FINAL DE TODA LA INSTRUCCIÓN “SELECT”, *excepto si usamos LIMIT en MariaDB o SQLite que iría antes del LIMIT.***
- Por defecto, se ordenan las columnas de forma ascendente (ASC) según los valores de la columna.

- Si se quiere ordenar por orden descendente se utiliza la palabra reservada “DESC” después del nombre del campo o del número del campo indicado en el **ORDER BY**.
- Cada campo ubicado en el **ORDER BY** se ordena de forma independiente del resto de campos.
- Se puede indicar un número en vez del nombre del campo, el valor indicado es la posición ordinal que dicho campo ocupa en el “SELECT” de la instrucción.
- Se pueden ordenar por distintos campos separados por comas, el orden de precedencia es de izquierda a derecha.

EJEMPLOS/EJERCICIOS:

○ Ejercicios (1)

- **SELECT** NIP, Nombre, edad
FROM Alumnos
ORDER BY Nombre, Edad

- **SELECT** NIP, Nombre, edad
FROM Alumnos
ORDER BY Edad **Asc**, Nombre **Desc**

- **SELECT** a.NIP, a. Nombre, A.edad as
Su_Edad
FROM Alumnos AS A
ORDER BY A.Edad **Desc**, a.Nombre

- **SELECT** NIP, Nombre, edad
FROM Alumnos
ORDER BY 2 **Desc**

- **SELECT** NIP, Nombre, edad as Edad_Alumno
FROM Alumnos
ORDER BY 1 **Desc**, EDAD

- **SELECT** A.Nombre as Nombre_Alumno
FROM Alumnos as A
WHERE A.Edad>27 **OR** A.Calificacion<25
ORDER BY A.Nombre **Desc**

- **SELECT** *
FROM Alumnos
WHERE Edad>30
ORDER BY 1 **Desc**, 2 **Asc**

- **SELECT** Alumnos.*
FROM Alumnos
WHERE Edad>20
ORDER BY 3 Desc, 4 Asc

- **SELECT** *
FROM Alumnos
WHERE calificacion **IS NULL**
ORDER BY 2,3,4

- **SELECT** *
FROM Alumnos
WHERE calificacion **IS NOT NULL**
ORDER BY 2,3,4

- **SELECT DISTINCT** Alumn.calificacion
FROM Alumnos AS Alumn
ORDER BY Alumn.calificacion Desc

- **SELECT DISTINCT** EDAD
FROM Alumnos
ORDER BY 1

- **SELECT DISTINCT** CALIFICACION, EDAD
FROM Alumnos
ORDER BY 1,2

- **SELECT DISTINCT** CALIFICACION, EDAD
FROM Alumnos
WHERE EDAD>30 **AND** CALIFICACION>6
ORDER BY 1,2

○ **Ejercicios (2)**

SQLite/MariaDB

```
SELECT Calificacion, Nombre FROM Alumnos ORDER BY 1 ASC LIMIT 3
```

```
SELECT Calificacion, Nombre FROM Alumnos ORDER BY 1 DESC LIMIT 5
```

```
SELECT Calificacion, Nombre FROM Alumnos ORDER BY 1 ASC LIMIT 2,3
```

```
SELECT Calificacion, Edad, Nombre  
FROM Alumnos  
ORDER BY 1 ASC, 2 DESC LIMIT 2,3
```

```
SELECT Calificacion FROM Alumnos ORDER BY 1 ASC LIMIT 3 OFFSET 2
```

```
SELECT * FROM Alumnos ORDER BY 1 LIMIT 5
```

```
SELECT ALL * FROM Alumnos ORDER BY 1 LIMIT 5
```

```
SELECT DISTINCT * FROM Alumnos ORDER BY 1 LIMIT 5
```

```
SELECT * FROM Alumnos LIMIT 4 OFFSET 2
```

```
SELECT * FROM Alumnos LIMIT 2,4
```

Material de apoyo
software:
bibliografía:
web:
otros